

How a contact is located

E-BTS · Tactile Lab, Nazarbayev University

WHAT IS BEING MEASURED

A soft elastomer pad carries a printed lattice of markers. An event camera watches it from below. Pressing the pad stretches its surface, so every marker near the contact is pushed AWAY from the contact point. The contact is therefore a SOURCE in the marker displacement field, and the estimator's whole job is to find where that source is.

Nothing here detects the indenter. It detects the deformation the indenter causes, which is why the sensor cannot tell an indenter from a fingertip of the same shape, and why it reports one contact position rather than a contact area.

THE PIPELINE, ONE EVENT WINDOW AT A TIME

- 1. ACCUMULATE** Events are collected into fixed windows of 40000 us. A window is reduced to the SET OF PIXELS that saw at least one event -- not a count, not a polarity. Markers are dark discs whose edges generate events as they move, so occupancy is what carries the shape.
- 2. DETECT** Blobs are found in that occupancy image and accepted as markers on radius and on FILL DENSITY -- the fraction of the blob's bounding disc that is actually occupied, which is what separates a marker from a smear. Expected radius comes from the pad geometry, 9.7 px here.
- 3. TRACK** Each detection is matched to a marker TRACK, and each track to a cell of the lattice. Once a baseline exists, detection is local: only a +-9 px window around each marker is searched, which is what keeps two neighbours 30 px apart from being confused for one another.

[!] Which centre that window follows is the defect this run exposed. It used to be the marker's REST site, so a marker displaced further than the window's half-diagonal could not be found at all -- see page 8.
- 4. BASELINE** With the pad unloaded, 20 windows (≥ 500000 us) of stable detections define each marker's REST POSITION. Everything downstream is a difference against that baseline, so a baseline captured under load defines the loaded shape as "undeformed" and silently offsets every later answer. This is checked, not trusted: see the tare test on page 4.
- 5. FIELD** Per cell (r, c): d = observed centre - baseline centre. Cells with no live marker are marked MISSING, not zero. A hole is not a displacement of zero, and treating it as one manufactures a gradient out of nothing.
- 6. DIVERGENCE** $\text{div} = d(dx)/dc + d(dy)/dr$ by central differences, in px per lattice cell. Any stencil touching a missing cell yields NaN, and a 1-cell border is masked. Outward flow gives a positive peak; the contact is at that peak.
- 7. LOCALISE** The two strongest peaks are found and must EARN their place (page 3). The winner's sub-cell position is refined, mapped to sensor pixels by a local plane fit through the neighbouring baseline sites, and then to millimetres through calibration/pixel_to_mm.json.

Stages 5-7 are shared with the offline analysis path (ml/contact_detect.py), which is why the constants below carry that file's names in comments.

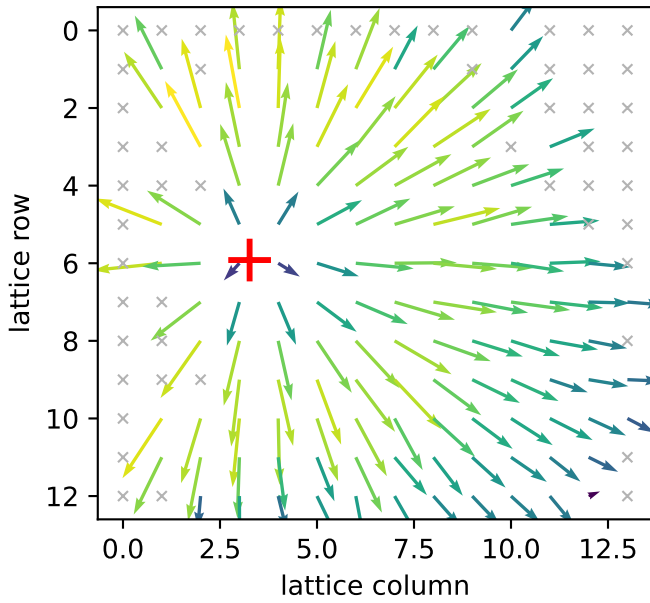
WHAT ONE WINDOW COSTS

The estimate is produced for every 40000 us window -- 211362 windows in this 2.2 hour run -- so "the contact position" is a time series at 25 Hz, not a

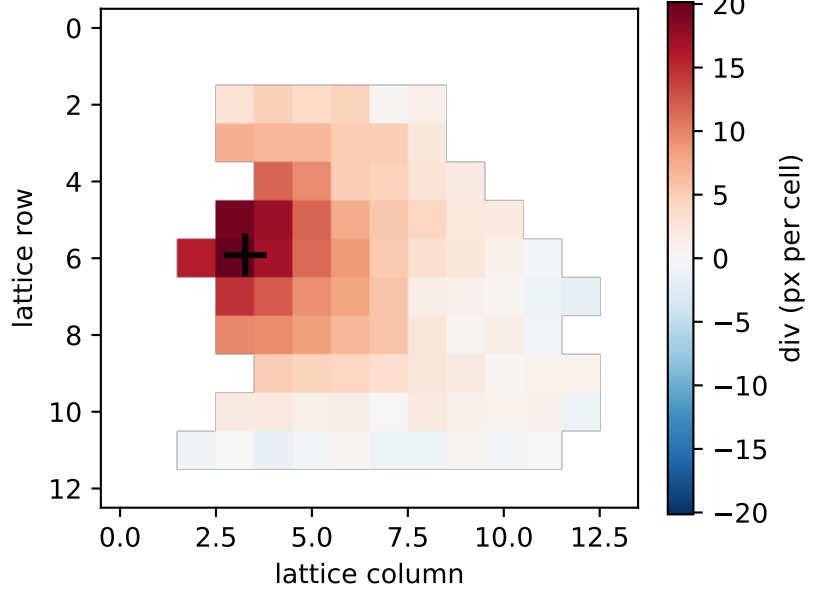
The method on real data: one 5 mm poke, one 1 mm poke

every panel is the estimator's own intermediate state, dumped from E_BTs_contact_replay

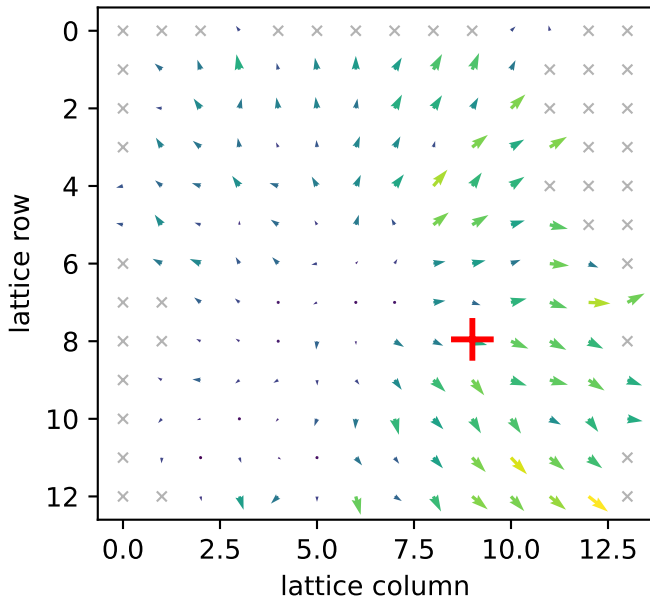
5 mm indent: markers pushed OUTWARD
(x = cell with no live marker)



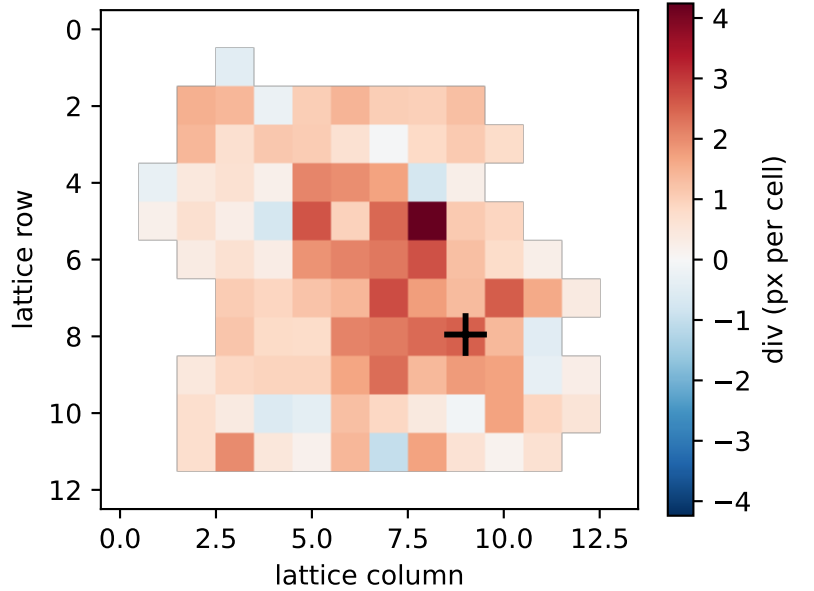
its divergence: peak 20.1 px/cell



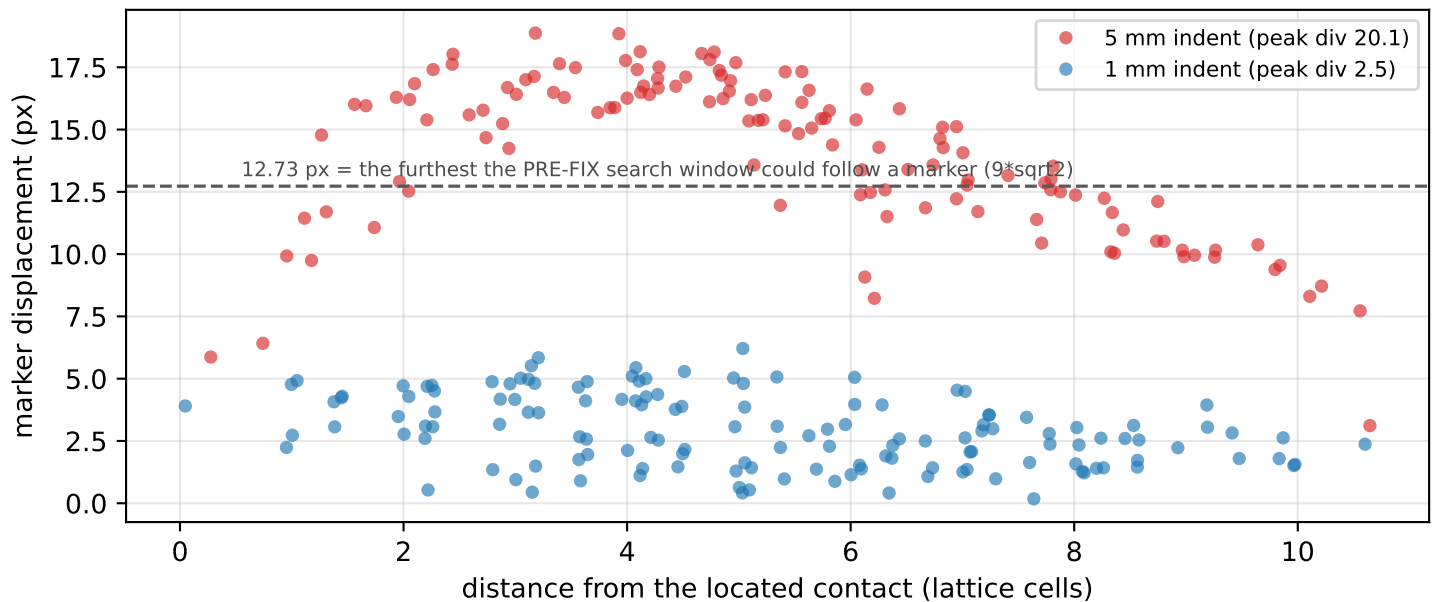
1 mm indent: the same field, far weaker



its divergence: peak 2.5 px/cell



Why depth helps: the signal is the displacement, and it grows with indentation



What must be true before a peak is called a contact

E-BTS · Tactile Lab, Nazarbayev University

A PEAK IS NOT YET A CONTACT

A peak finder asked for N peaks returns N peaks; it cannot decline. Given a broad noisy plateau it returns the N highest points on it. So candidates are tested, and a run that produces no estimate is a result rather than a failure.

Two candidates are always sought, never one. The second is used only to decide whether the first is ALONE: reporting the stronger of two contacts as though it were the only one is the failure this catches, and it is why the log carries an `ambiguous` flag beside `valid`.

THE FOUR GATES, IN THE ORDER THEY ARE APPLIED

SEPARATION	≥ 3.0 cells between the two candidates. Two peaks on one lobe are one contact reported twice.
COHERENCE	≥ 0.60 How consistently the surrounding markers point AWAY from the candidate. A real contact pushes outward in every direction; a tracking glitch pushes one way. Calibrated against real runs by ml/contact_detect.py, not chosen here.
RESOLVABILITY	≥ 0.15 The saddle between two candidates must be at least this far below the weaker of them, or they are one broad lobe rather than two contacts.
DIVERGENCE	≥ 1.5 px per cell [!] THE ONE THAT IS NOT CALIBRATED Below this, not a contact. Unlike the three above, this was set by judgement -- low enough not to miss a light touch, high enough that an unloaded pad's tracker jitter should not reach it. Page 6 measures what it actually costs, which is the first time it has been checked rather than assumed.

A candidate that clears all four still needs a POSITION, and that can fail independently: the local plane fit in the next section needs at least 6 neighbouring cells with live markers. Near the edge of the tracked field, or in a region where the lattice has holes, it will not have them.

FROM A CELL INDEX TO MILLIMETRES

The peak arrives as a fractional cell (row, col) -- for the 5 mm poke opposite, (5.92, 3.26) of a 13 x 14 lattice. Converting it:

cell -> pixel	a plane is least-squares fitted to the BASELINE pixel positions of the cells within 2 cells of the peak, and evaluated at the fractional peak. Using baseline rather than current positions is deliberate: the answer wanted is where the contact is on the UNDEFORMED pad, not where the stretched markers currently are.
pixel -> mm	calibration/pixel_to_mm.json, fitted by franka/calib_raster.py against the robot's own XY. Output is ROBOT BASE millimetres, so comparing against the Franka is a subtraction with no frame conversion in between that could be silently wrong by a sign or a rotation. The C++ and Python evaluations of this map agree to under 1 um across the sensor (ml/check_pixel_to_mm_port.py).

For that poke the chain ran cell (5.92, 3.26) -> pixel (276.6, 220.3) -> the millimetres reported in the log, with 129 of 182 lattice sites carrying a live marker at that instant.

Accuracy, part 1: does it detect the contact at all

E-BTS · Tactile Lab, Nazarbayev University

TWO DIFFERENT QUESTIONS, REPORTED APART

DETECTION when the pad IS being touched, does a contact get reported, and
when it is NOT, does the estimator correctly report nothing?
LOCALISATION given that a contact was reported, how far is it from the truth?

They fail independently and a single "accuracy" number hides both. What follows is detection; page 5 is localisation.

DETECTION RATE BY ROBOT PHASE			211362 windows, 1094 of 1100 pokes
phase	windows	reported	

tare	27609	5.5%	
dip	73759	77.5%	
dwel	55131	89.3%	
retract	30627	24.9%	
travel	23886	5.7%	

`tare` is a ~1 s hold with the indenter deliberately clear of the pad, run before every single poke. `dwel` is the part of a poke where the indenter is stationary and fully engaged. Those two rows are the false-positive and true-positive rates of the detector, measured 1100 times each over 2.3 hours.

The remaining phases are motion -- the indenter approaching, retracting or travelling. They are neither clean positives nor clean negatives (during part of `dip` the indenter is genuinely in contact), so they are listed but should not be read as error rates.

[!] THE TARE TEST IS THE ONE THAT MATTERS MOST, AND IT IS FREE

Every estimate is a difference against the marker baseline. If that baseline drifts, or was captured under load, every coordinate this sensor reports is offset by an amount nothing downstream can detect or recover. A rising false-contact rate through a run is the signature, so it is split in half and compared:

first half 5.78% -> second half 5.19% no drift

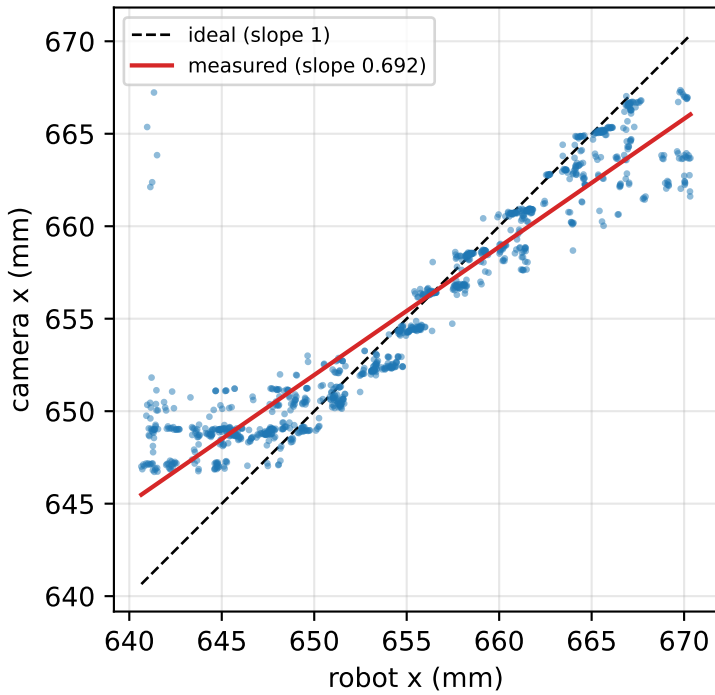
Over a 2.3-hour run this is the single most likely way for the measurement to be quietly wrong, and it costs nothing to check.

WHAT THE FAILURES LOOK LIKE

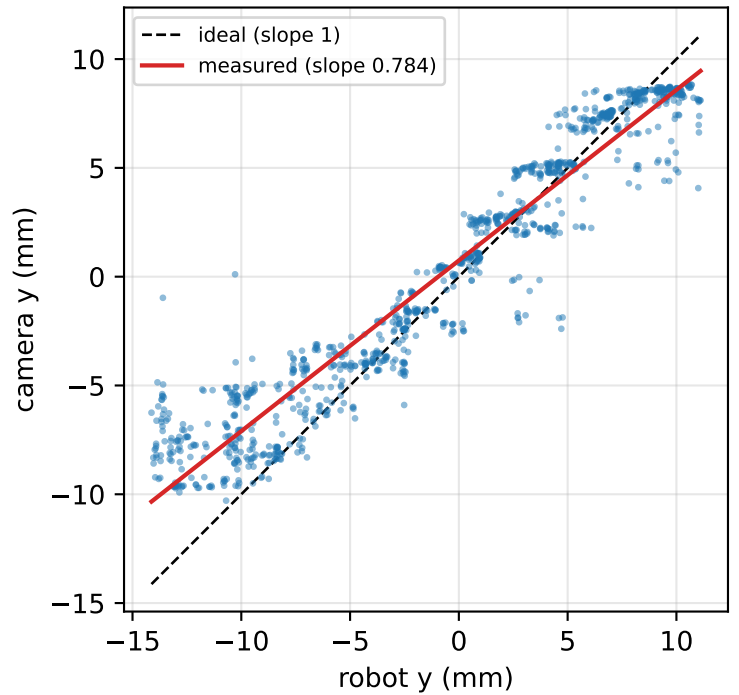
Of 1100 pokes, 6 produced fewer than 3 usable single-contact windows in their dwell and were dropped. They are not spread evenly across depth -- shallow pokes fail more often, because the divergence lobe is weaker and closer to the threshold. That is a bias, not a random loss, and it means any error statistic computed over the survivors is mildly optimistic.

Accuracy, part 2: where it says the contact is

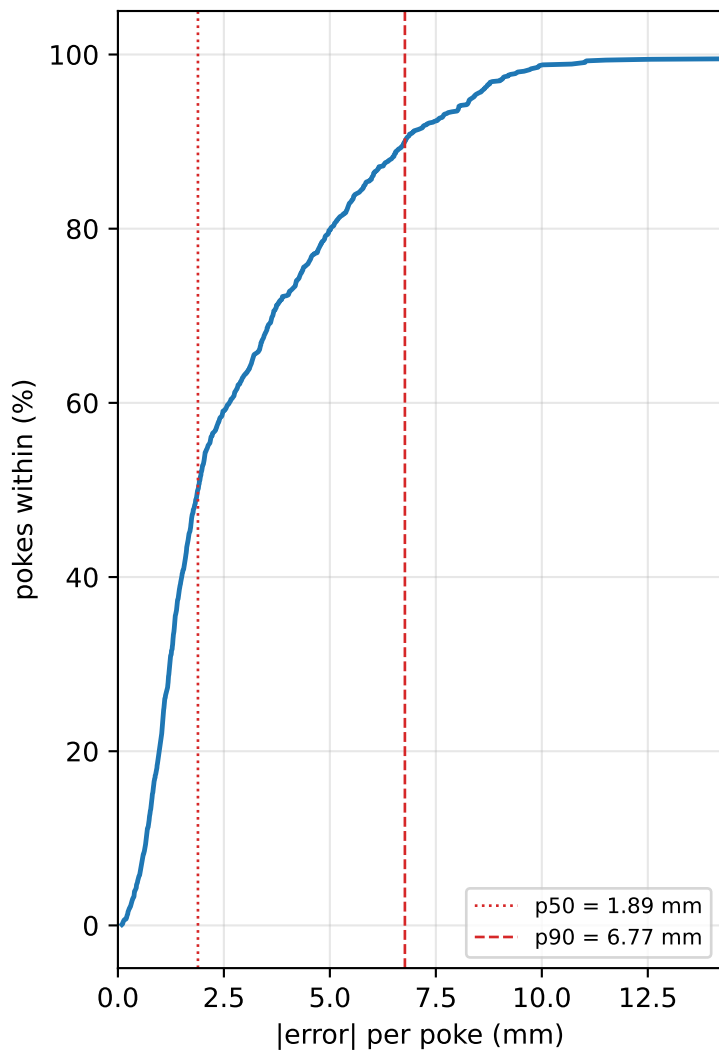
x: gain 0.692, $r = 0.939$



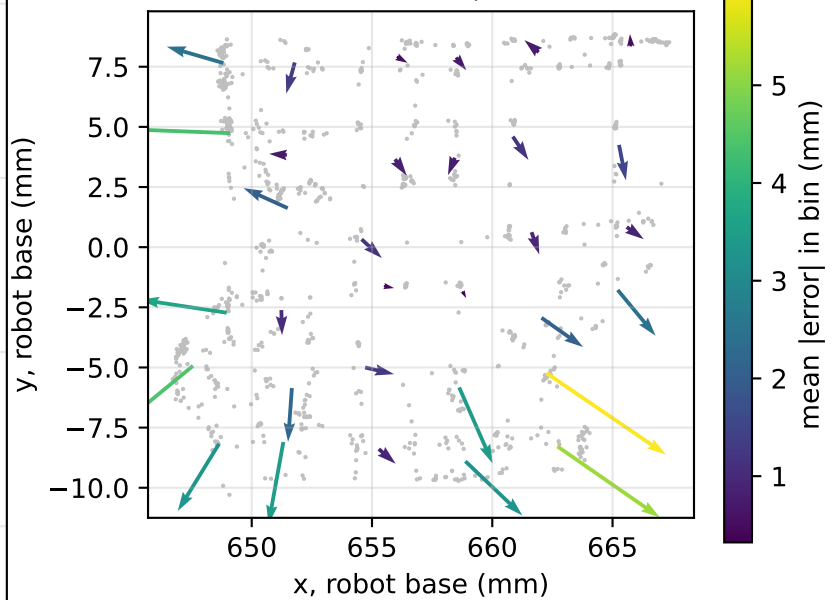
y: gain 0.784, $r = 0.962$



Error distribution (n = 1094)

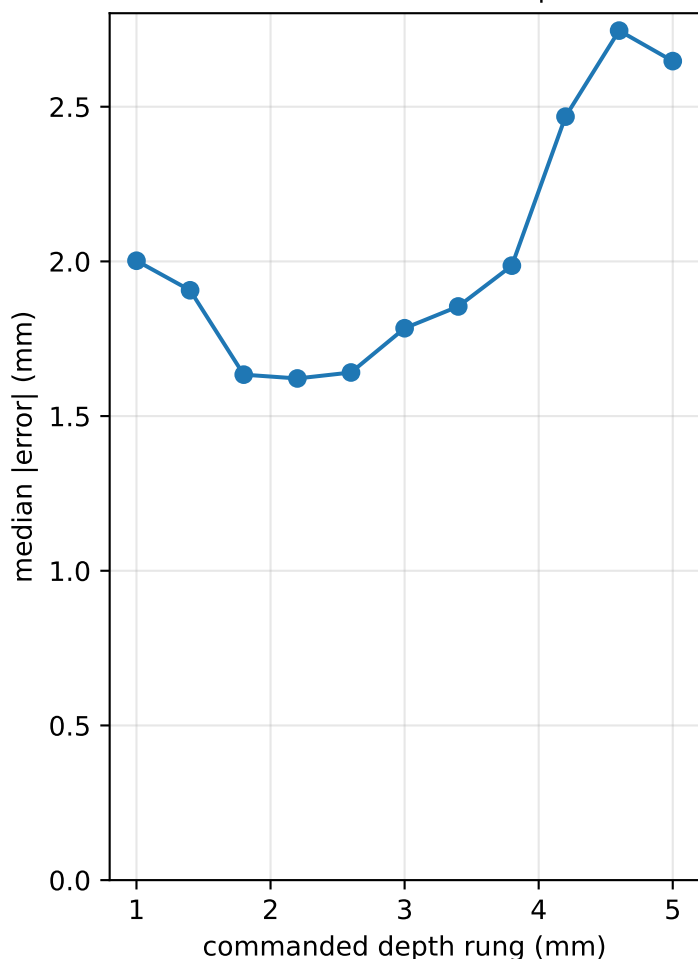


Mean error per region, true scale
estimate -> truth, 1084 pokes in 32 bins

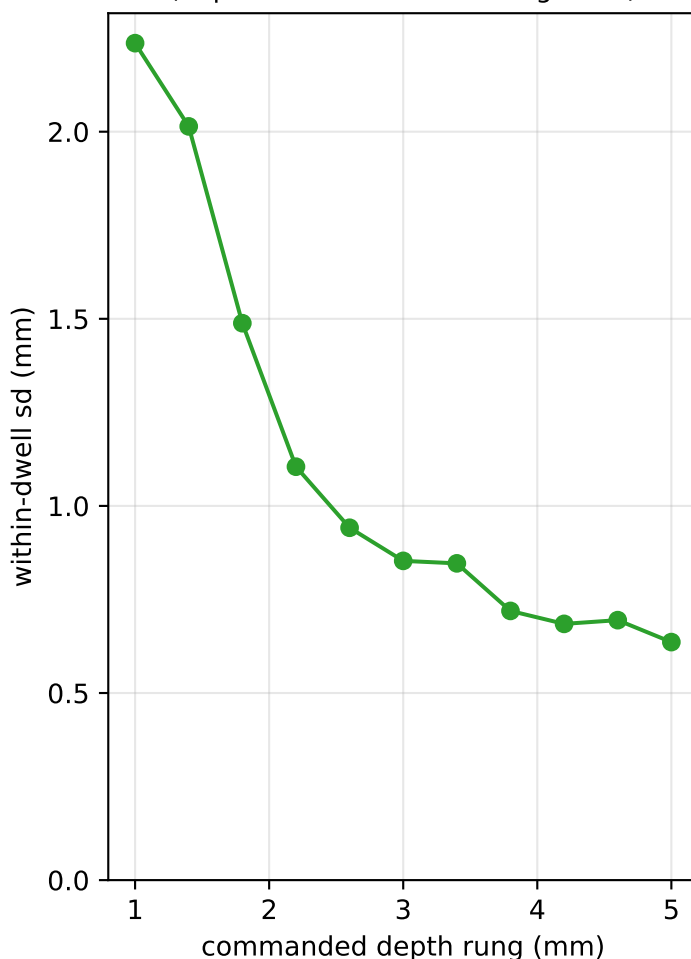


Accuracy vs depth, and what the divergence threshold costs

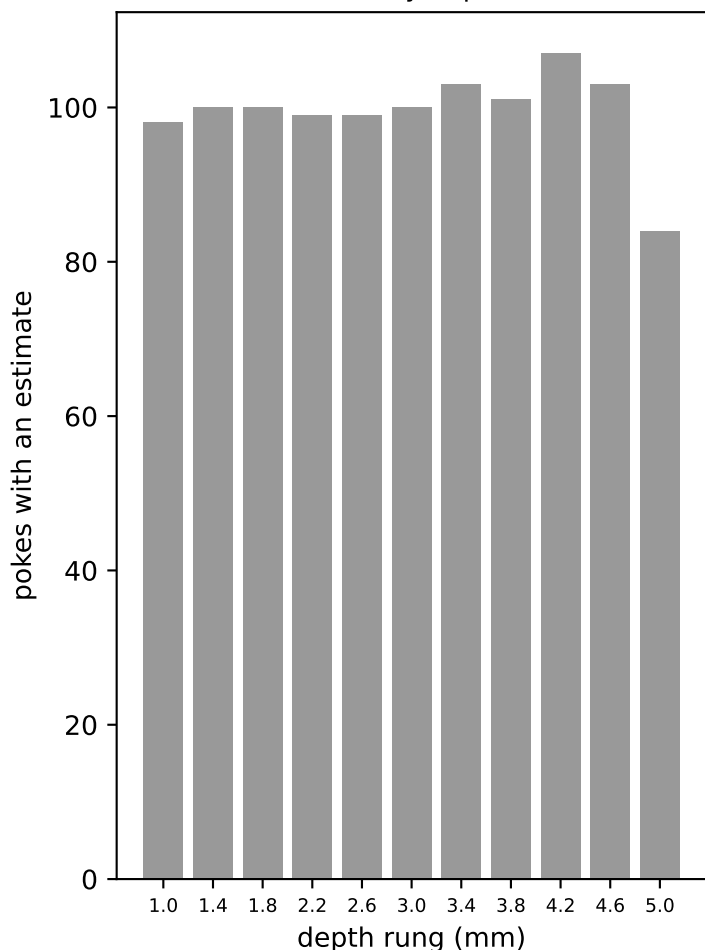
Localisation error vs depth



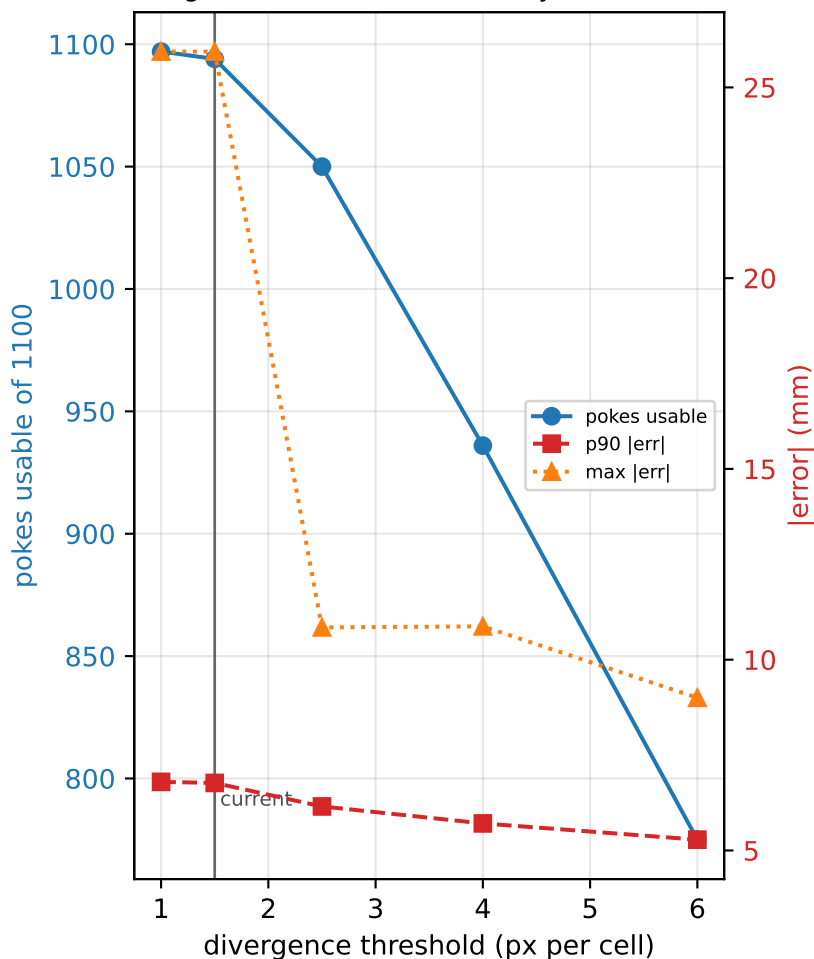
Repeatability vs depth (improves as the lobe strengthens)



Yield by depth



Raising the threshold: what it buys, what it costs



Accuracy, part 3: the numbers and what limits them

E-BTS · Tactile Lab, Nazarbayev University

THE HEADLINE

pokes	1094 of 1100
bias (mean offset)	dx -0.492 dy -0.854 mm bias 0.985 mm
scatter about it	sd 3.294 mm (x) 2.279 mm (y)
error	median 1.888 mean 3.010 p90 6.770 max 25.939 mm
within-dwell spread	median 1.029 mm

Read the SCATTER, not the bias. A constant offset is a wrong origin or a stale calibration: it is correctable after the fact and it is not the sensor. The spread about it is the estimator, and it is not correctable.

Read the MEDIAN, not the mean. The error distribution has a tail of weak-peak outliers (page 6 shows the threshold that removes it); the mean follows the tail, the median describes the typical poke.

[!] THE GAIN, AND WHY IT LIMITS EVERYTHING ABOVE

The estimate tracks the indenter closely -- $r = 0.939$ in x, 0.962 in y -- but it TRAVELS TOO LITTLE. Move the indenter 1 mm and the reported contact moves 0.692 mm in x and 0.784 mm in y.

That is a systematic, position-dependent error, so the error statistics above are dominated by it: a poke near the centre of the pad is reported almost correctly and one near the edge is reported several millimetres inboard. They therefore describe THIS DEFECT more than they describe the estimator, and must not be quoted as the sensor's localisation accuracy.

What IS trustworthy is the within-dwell repeatability -- how much the estimate moves while the indenter is held still -- because the gain error does not affect it: median 1.029 mm, improving with depth (page 6).

THE CONTROL THAT MAKES THIS COMPARABLE

	gain x	gain y	median e
live contact.csv (pre-fix, as run)	0.598	0.700	3.001 mm
replay --legacy-search (CONTROL)	0.637	0.719	2.727 mm
replay, fixed	0.712	0.790	1.826 mm

paired over the 890 pokes present in all three

The middle row is the point. Replaying the PRE-FIX code over the same recording reproduces what the run itself logged, which is what licenses attributing the third row's improvement to the code change rather than to a luckier set of pokes. Without that row the comparison would be between two different samples and would mean very little.

It also bounds the resolution of this comparison: the control disagrees with the live run by about 6% in gain, from the window-phase offset and from the replay building its own baseline. Differences smaller than that should not be read as real.

THE DIVERGENCE THRESHOLD, MEASURED FOR THE FIRST TIME

threshold	pokes	median	p90	max	
1.00	1097	1.898	6.798	25.939	
1.50	1094	1.888	6.770	25.939	<- current
2.50	1050	1.799	6.154	10.846	
4.00	936	1.696	5.709	10.874	
6.00	775	1.579	5.283	9.002	

Raising 1.5 -> 2.5 costs 4% of pokes and removes the entire tail: the worst error falls from 25.9 mm to 10.8 mm. The outliers are weak peaks, and 1.5 is low enough to admit them. On this evidence 1.5 is not the right operating point -- but it is one run on one pad, and the gate was never the reason pokes were being LOST (that was the search window defect, now fixed).

Limits, and what would settle what remains

E-BTS · Tactile Lab, Nazarbayev University

WHAT THIS DOCUMENT DOES NOT ESTABLISH

1. A HEADLINE ACCURACY FIGURE. The gain defect above is systematic and large, so the millimetre figures characterise the defect, not the sensor. A real localisation accuracy needs the gain at or near 1.0 first.
2. THAT THE GAIN IS UNDERSTOOD. One cause was found and fixed. Markers are detected by searching a small window around each one, and that window used to follow the marker's REST position while the marker itself moved 15-25 px under a 5 mm indent -- so exactly the markers carrying the strongest signal fell outside their own search window and were lost. The peak was then computed from a truncated ring, which the edge of the tracked field clips asymmetrically for any off-centre poke, pulling the estimate inward.

The fix -- follow the marker, fall back to its baseline site when it is lost -- is verified to raise the largest followable displacement from 12.73 px (exactly $9\sqrt{2}$), the diagonal of the old search square, which is what confirms the mechanism) to 45.18 px. It cuts the median error by a third and takes the pokes yielding an estimate from 914/1100 to 1094/1100.

It does NOT bring the gain to 1.0. Against the replayed pre-fix control the compression ($1 - \text{gain}$) falls from 0.363 to 0.288 in x and from 0.281 to 0.210 in y -- between a fifth and a quarter of it removed, and the rest unexplained. Note also that the mean BIAS grew ($|\text{bias}|$ 0.275 $\rightarrow$ 0.985 mm) while the scatter shrank: the fix moved the estimate, and not every statistic moved the same way.

3. THAT THE REMAINING COMPRESSION IS ONE EFFECT. Candidates not yet separated: the elastomer's surface shear may genuinely under-report the indenter's travel at the pad surface; the divergence peak of a truncated or asymmetric lobe is biased toward the field centre; and the lattice itself is sampled only every ~30 px, so a sub-cell fit is doing real work near the edges.
4. ANY PAD BUT THIS ONE. One elastomer, one marker lattice, one calibration, one run. Marker pitch, thickness and stiffness all enter the displacement field directly.

WHAT WOULD SETTLE IT, IN ORDER OF COST

- Re-measure with the OFFLINE estimator (ml/blue_circle_grid), which re-detects markers over the whole image and never had the search-window defect. If its gain is also below 1 on this recording, the cause is physical or geometric rather than a tracking failure. No robot time: the recording is on disk.
- Sweep the divergence threshold and re-fit the gain at each value. If the gain depends on the threshold, the peak is being biased by which markers survive. Also free -- one replay pass already supports it.
- Only then, a fresh campaign. It costs ~2.8 h of arm time and ~66 GB, and on the evidence here it would return a gain near 0.7-0.8 rather than 1.0, so it would not yet yield the accuracy figure it is meant to produce.

REPRODUCING THIS DOCUMENT

```
./build/E_BTS_contact_replay --raw recordings/<run>/camera.raw \  
  --time-ref recordings/<run>/contact.csv \  
  --out recordings/<run>/contact_replay.csv --accum-us 40000  
python3 ml/contact_error.py recordings/<run> --contact contact_replay.csv  
python3 ml/contact_method_report.py recordings/<run>
```

See docs/CONTACT_REPLAY.md for the replay's own caveats -- in particular three clock traps that each produced entirely plausible but wrong output before they were found.